

Praktikumsaufgaben

Inhaltsverzeichnis

Assembler-Aufgaben	2
Aufgabe 1	2
Aufgabe 2	2
Aufgabe 3	3
Aufgabe 4	3
Aufgabe 5 (<i>optional</i>)	3
Aufgabe 6	4
Aufgabe 7	4
Aufgabe 8	4
Aufgabe 9	5
Aufgabe 10	5
Aufgabe 11 (<i>optional</i>)	5
Aufgabe 12	6
Aufgabe 13 (<i>optional</i>)	6
Aufgabe 14 (<i>optional</i>)	6
C-Aufgaben	7
Aufgabe 1	7
Aufgabe 2	7
Aufgabe 3	8
Aufgabe 4	8
Aufgabe 5	9
Aufgabe 6	10
Aufgabe 7	10
Aufgabe 8	11
Aufgabe 9	11

Assembler-Aufgaben

Aufgabe 1

Schreiben Sie ein Assembler-Programm, welches das 32-bit Allzweckregister R0 des Micro-Controllers nacheinander mit Daten füllt und diese in den Speicher ab Adresse 0x20000000 schreibt. Für die Speicheradresse können Sie das Register R1 verwenden. Verwenden Sie die Daten-Worte 0x11111111, 0x22222222, 0x33333333, 0x00000000. Debuggen Sie das Programm wie beschrieben Schritt für Schritt und sehen Sie sich die Inhalte der Register und des Speichers ab Adresse 0x20000000 während der Ausführung an. Verwenden Sie zur Anzeige des Speichers am besten die Anzeigeoption *Hex Integer*.

Aufgabe 2

In einer Tabelle direkt hinter dem Programmcode stehen die folgenden Zahlen:
0x11111111, 0x22222222, 0x33333333, 0x44444444, 0x55555555, 0x66666666
0x77777777, 0x88888888, 0x99999999, 0xAAAAAAAA, 0x00000000.

Schreiben Sie ein Programm, das die Tabelle in den Speicher ab Adresse 0x20000000 umkopiert. Das Programm soll beendet werden, wenn ein Tabellenelement 0x00000000 gefunden wird.

- a) Die Zahl 0x00000000 soll mit kopiert werden.
- b) Die Zahl 0x00000000 soll nicht mit kopiert werden.
- c) Als Endekennung soll nun nicht die 0x00000000 sondern die Zahl 0x0F0F0F0F verwendet werden. Diese soll mitkopiert werden.
- d) Die Tabelle soll auf fortlaufenden Adressen abgelegt werden, bis die Adresse 0x20000100 erreicht ist. Wenn die Tabelle mehrfach in den Bereich passt, soll sie auch mehrfach kopiert werden.

Aufgabe 3

Schreiben Sie ein Programm, das die Zahlen der folgenden Tabelle addiert ($a + b$). Bestimmen Sie jeweils vorher qualitativ das Ergebnis der Rechnung (große/kleine Zahl, positiv/negativ) und die auftretenden Flags.

a	b	Länge	Ergebnis	gültig?	N	Z	C	V
0x00000000	0x00000010	32 Bit						
0xFFFFFFFF	0xFFFFFFFFE	32 Bit						
0x7FFFFFFF	0x7FFFFFFF0	32 Bit						
0x80000000	0x80000001	32 Bit						
0xFFFFFFFF	0x00000001	32 Bit						
0x88888882	0x88888883	32 Bit						
0x00000005	0x1111111A	32 Bit						

Die Zahlen stehen als Tabelle direkt hinter dem Programmcode. Verwenden Sie für den Zugriff auf die Tabelle die Adressierungsart *indexed postincrement*.

- Die Ergebnisse sollen ab Adresse 0x20000000 im Speicher stehen.
- Prüfen Sie im Programm, ob ein Overflow (Zahlenbereichsüberschreitung) auftritt. Die Ergebnistabelle (ab 0x20000000 im Speicher) soll die Strukturierung aufweisen:
Zahl1, Zahl2, Ergebnis, Overflow

Aufgabe 4

Schreiben Sie ein Programm, welches die folgenden 64-bit Zahlen addiert:

Zahl1: 0x0182A02481C2348F

Zahl2: 0x02C08300F19310C0

Aufgabe 5 (optional)

Schreiben Sie ein Programm, welches die Zahlen einer Zahlenreihe (32-bit Zahlen) von 1 bis zur Obergrenze MAX addieren kann. Das Ergebnis steht an der Speicheradresse 0x20000000. Zur Erzeugung der Einzelelemente dieser Zahlenreihe wird die Konstante STEP benutzt (neues Reihenelement = altes Reihenelement + STEP). Wenn ein Overflow auftritt, soll die Fehlerkennung 0xFFFF an die Speicheradresse 0x20000000 gespeichert werden. Beispiel: Für MAX = 8 und STEP = 2 werden die Zahlen 1, 3, 5, 7 addiert. Das Ergebnis an der Speicheradresse 0x20000000 ist die Zahl 16. Nutzen Sie den Speichermonitor der IDE, um die Summe zu überprüfen.

Aufgabe 6

Die rechte Leuchtdioden-Reihe (rot) ist an Pin 7 bis 0 des GPIOC-Registers des Mikrocontrollers angeschlossen. Eine LED leuchtet, wenn am entsprechenden Pin ein 0-Signal ausgegeben wird. Ermitteln Sie aus dem Reference Manual des verwendeten Mikrocontrollers die Basisadresse für die GPIOC-Register sowie den Offset für das ODR (Output Data Register). Legen Sie mithilfe der `.equ`-Assembler-Direktive Symbole für die entsprechenden Adressen an, um diese im Programm über ihren Namen verwenden zu können. Prüfen Sie außerdem in der CubeMX Perspektive, ob die entsprechenden Pins korrekt als Output konfiguriert sind.

- a) Auf der rechten Leuchtdioden-Reihe soll ein Binärzähler dargestellt werden (0x00, 0x01, . . . , 0xFE, 0xFF, 0x00, 0x01, . . .). Falls das Ergebnis nicht zufriedenstellend ist, programmieren Sie eine Warteschleife nach jeder Ausgabe auf die LEDs.
- b) Die linke Leuchtdioden-Reihe ist an Pin 15 bis 8 des GPIOC-Registers des Mikrocontrollers angeschlossen. Hier soll der gleiche Zähler dargestellt werden, allerdings muss hierfür die Ausgabe invertiert werden. Eine LED leuchtet also, wenn hier ein 1-Signal ausgegeben wird.

Aufgabe 7

Auf den Leuchtdioden-Reihen soll ein 16-Bit-Binärzähler dargestellt werden (0x0000, 0x0001, 0x0001, . . . , 0xFFFE, 0xFFFF, 0x0000, 0x0001, . . .). Die Schalter sind an GPIOB, Pin 15 bis 8 angeschlossen. Ermitteln Sie aus dem Reference Manual des verwendeten Mikrocontrollers die Basisadressen für die GPIO-Register sowie den Offset für das ODR (Output Data Register) bzw. das IDR (Input Data Register).

- a) Programmieren Sie den Zähler wie beschrieben. Falls das Ergebnis nicht zufriedenstellend ist, programmieren Sie eine Warteschleife nach jeder Ausgabe auf die LEDs.
- b) Die Anzeige des Zählers soll abgebrochen werden, wenn einer der Schalter geschaltet wird (alle LEDs sollen dann ausgeschaltet werden).
- c) Erweitern Sie das Programm: Bei jeder Veränderung der Schalterstellung soll die Ausgabe angehalten werden, bei der nächsten Veränderung läuft die Ausgabe weiter.

Aufgabe 8

Programmieren Sie ein Lauflicht über 16 LEDs. Das Lauflicht soll von rechts nach links über die LED-Reihen laufen.

Aufgabe 9

Schreiben Sie ein Programm, das nach einer Änderung der Schalterstellung zwischen der Ausgabe des 16 Bit Zählers und der Ausgabe des 16 Bit Lauflichts umschaltet.

- a) Der Zähler bzw. das Lauflicht beginnen nach jedem Umschalten in der Ausgangsstellung.
- b) Der Zähler bzw. das Lauflicht werden nach jedem Umschalten bei dem Zählerstand (bei der Lauflichtposition) weitergeführt, bei der sie verlassen wurden.

Aufgabe 10

Aus einer byteweise aufgebauten Tabelle sollen Werte geholt und in einer Endlosschleife auf der linken LED-Reihe ausgegeben werden. Es wird der Wert ausgegeben, dessen Tabellenplatz mit Hilfe der Schalter an GPIOB, Pin 8 bis 5 bestimmt wird.

Platz	0	1	2	3	4	5	6	7
Wert	0xFF	0x05	0xC8	0xEC	0x4A	0x78	0xF0	0x18
Platz	8	9	10	11	12	13	14	15
Wert	0x24	0xE9	0x7A	0x85	0x48	0x64	0x36	0x1F

Aufgabe 11 (*optional*)

Aus der folgenden Matrix soll der Wert (Byte) in einer Endlosschleife ausgegeben werden, dessen Zeile durch die Schalter an Pin 8, 9 und 10 und dessen Spalte durch die Schalter 13, 14, und 15 bestimmt ist.

	0	1	2	3	4	5	6	7
0	0xC2	0x58	0x7A	0x71	0x92	0x37	0x47	0x25
1	0xA1	0xE6	0x75	0x84	0x24	0xED	0x82	0xC1
2	0x8C	0x1A	0x62	0xDC	0xFF	0x18	0x42	0x00
3	0xA4	0x35	0x28	0x73	0x32	0x73	0xA5	0xC6
4	0x34	0x74	0x23	0xAE	0xDC	0x54	0xEA	0x32
5	0xCE	0x34	0x45	0x67	0x87	0x34	0x76	0x54
6	0xED	0x21	0xAE	0xED	0x57	0x43	0x23	0xED
7	0xE4	0x34	0x78	0x24	0x42	0x46	0x64	0x47

Aufgabe 12

Wie in Aufgabe 7a) soll ein 16-Bit-Binärzähler programmiert werden. Dabei sollen diesmal aber die folgenden Programmteile als Unterprogramme realisiert werden:

- Initialisierung des Peripherie-Bausteins
- Erzeugen der auszugebenden Zahl
- Ausgeben der Zahl auf die LED-Reihe
- Warteschleife

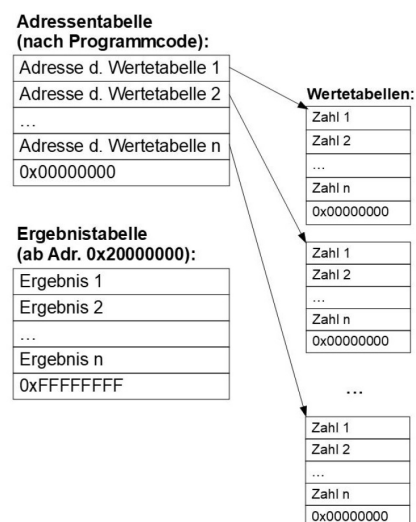
Das Hauptprogramm besteht aus einer Schleife, die fast ausschließlich Unterprogramm-Aufrufe enthält. Überlegen Sie sich, welche Assembler-Befehle sinnvollerweise nicht in Unterprogramme ausgelagert werden sollten. Arbeiten Sie das Programm in Einzelschritten ab (nicht die Warteschleife!) und achten Sie besonders auf den Stackpointer und den Stackinhalt.

Aufgabe 13 (*optional*)

Erweitern Sie das Programm aus Aufgabe 12. Zusätzlich soll vor dem Wechsel auf die nächste Zahl die LED-Reihe mit dem alten Wert 4 mal blinken. Für die verschiedenen Wartezeiten soll dasselbe Unterprogramm verwendet werden.

Aufgabe 14 (*optional*)

Ein Programm soll die negativen Zahlen in mehreren Werte-Tabellen zählen und die Ergebnisse (Anzahl der negativen Zahlen) in einer Ergebnis-Tabelle ablegen. Die Werte-Tabellen enthalten vorzeichenbehaftete 32-Bit Zahlen (die Null ist hier keine gültige Zahl und tritt nur als Endekennung auf). Im Speicher steht nach dem Programm eine Adressentabelle. Die Adressentabelle enthält die Zeiger auf die Werte-Tabellen. Alle Wertetabellen enthalten als Endekennung die Zahl 0 (jeweils in der Länge der Tabellenobjekte). Die Ergebnistabelle soll im Speicher ab der Adresse 0x20000000 stehen. Sie soll nach Abarbeitung des Programms 64 Bit Zahlen enthalten. Jedes Tabellenelement ist das Zählergebnis der negativen Zahlen einer Wertetabelle. Nach dem letzten Zählergebnis ist die Endekennung 0xFFFFFFFF in die Tabelle zu schreiben. Schreiben Sie ein Programm, das die negativen Zahlen in der oben beschriebenen Weise zählt und die Ergebnisse speichert.



C-Aufgaben

Aufgabe 1

Zunächst sollen einige Variablen auf der Konsole ausgegeben werden.

- a) Schreiben Sie ein Programm, das eine Start-Meldung ausgibt und danach in einer Schleife einige Variablen hochzählt und ausgibt. Legen Sie die Variablen als Modul-Variablen an.
 - Integer-Variable `count`, Anfangswert 0
 - Character-Variable `buchstabe`, Anfangswert `0x41`
 - Float-Variable `floatwert`, Anfangswert `0.0`
- b) Ändern Sie Ihr Programm von Aufgabenteil a) so, dass die Variable `count` von 1 bis 20 hochgezählt wird und danach das Programm mit der Start-Meldung von vorne beginnt.

Tipp: Sie können die Funktion `HAL_Delay(uint32_t Delay)` dazu nutzen, eine Wartezeit festzulegen. Binden Sie dafür die Datei `stm3214xx_hal.h` in Ihr Programm ein.

Aufgabe 2

Nun soll mit Feldern gearbeitet werden.

- a) Legen Sie ein Integer-Feld mit 10 Elementen an und füllen Sie das Feld in einer Schleife durch Tastatureingaben. Geben Sie danach das Feld aus.
- b) Erweitern Sie Ihr Programm durch eine Tendenzanzeige während der Eingabe: die Zahl ist größer/kleiner/gleich gegenüber dem letzten Eingabewert.
- c) Fügen Sie eine Tendenzanzeige ein, die sich auf den bisherigen Mittelwert bezieht: die Zahl ist größer/kleiner als der Mittelwert aller bisher eingegebenen Zahlen.

Aufgabe 3

Legen Sie ein statisches Feld von Strukturen (10 Feldelemente) als Modul-Variable an.

- a) Füllen Sie die Struktur mit Werten, die Sie von der Tastatur einlesen. Geben Sie die Werte aus, nachdem das gesamte Feld gefüllt ist. Beobachten Sie das Füllen der Strukturen im Debugger.
- b) Legen Sie die 10 Strukturen als Ringpuffer (Feld!) an und geben Sie die letzten 10 Einträge aus. Als Ereignis zum Abbruch der Messwertaufnahme wählen Sie den Eingabewert 99.

Strukturelemente:

```
short int laufende_nummer; short int messwert; char beschreibung[20];
```

Aufgabe 4

Erzeugen Sie einen neuen Datentyp (Struktur-Prototyp) mit den Strukturkomponenten:

```
short int laufende_nummer;  
short int maschine;  
short int sensor;  
short int messwert;  
char beschreibung[10];
```

- a) Fordern Sie Speicher für 4 Objekte dieses Datentyps an mit der Library-Funktion `malloc`. Füllen Sie diese Strukturen mit Werten (Zuweisungen, kein `scanf`). Beobachten Sie das Füllen der Strukturen im Debugger.
- b) Legen Sie 10 Objekte des gleichen Datentyps hintereinander in einen Speicher, der auf der festen Adresse `0x20000000` beginnt. Füllen Sie die Strukturen mit Werten (Zuweisungen, kein `scanf`). Beobachten Sie das Füllen der Strukturen im Debugger.
- c) Füllen Sie die Struktur so mit Dummy-Elementen auf, dass Sie bei der Programm-entwicklung den Inhalt im Datenfenster des Debuggers möglichst leicht überblicken können.

Aufgabe 5

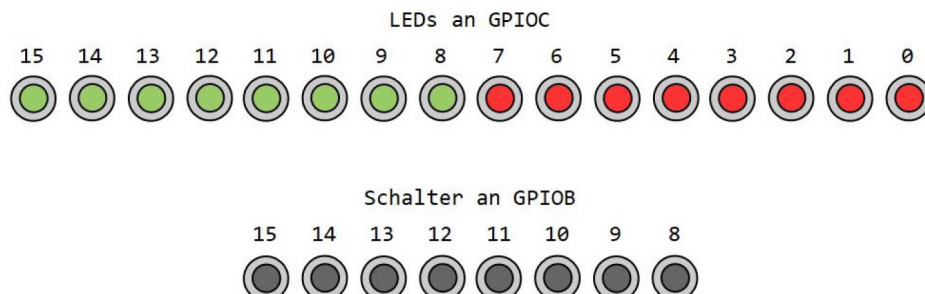
In einer Endlosschleife sollen die Schalter eingelesen und auf den LEDs ausgegeben werden. Nehmen Sie als Beispiel die Folien zur Vorlesung (Peripherie, GPIO_X). In den Vorlesungsunterlagen finden Sie den Struktur-Prototypen:

```
typedef struct
{
    long  gpiox_moder;    // 0
    long  gpiox_otyper;   // 4
    long  gpiox_ospeedr;  // 8
    long  gpiox_pupdr;    // C
    long  gpiox_idr;      // 10
    long  gpiox_odr;      // 14
    long  gpiox_bsrr;     // 18
    long  gpiox_lckr;     // 1C
    long  gpiox_afrl;     // 20
    long  gpiox_afrh;     // 24
    long  gpiox_brr;      // 28
    long  gpiox_ascr;     // 2C
} GPIO_X;
```

Diesen Struktur-Prototypen können Sie für alle GPIO-Register verwenden. Denken Sie zudem daran, den Input der Schalter mit einer Maske zu verunden, um ausschließlich die von den Schaltern belegten Pins einzulesen. Die Ausgabe auf den LEDs können sie dann mit einem Logical Shift realisieren. Denken Sie zudem daran, dass die roten LEDs invertiert sind (eine LED leuchtet, wenn an dem jeweiligen Pin eine 0 ausgegeben wird).

Maske für das Einlesen der Schalter:

```
#define MASK_B 0xFF00
```



Aufgabe 6

Das Programm soll die Schalterstellungen der Kippschalter einlesen. Bei jeder Veränderung der Schalterstellung soll der eingelesene Wert abgespeichert werden. Falls das höchstwertige Bit auf 1 schaltet, soll das Einlesen beendet und die vorhandenen Werte ausgegeben werden. Denken Sie daran, dass die Schalter nicht entprellt sind.

- a) Legen Sie zum Abspeichern der Werte einen Ringspeicher ab Adresse `0x20000000` an. Der Ringspeicher soll aufgebaut sein als einfach verkettete Liste aus 10 solchen Strukturen. Jeder Eintrag wird durch eine Struktur dargestellt und besteht aus:
 - Pointer zum nächsten Listenelement
 - Nummer der Messung
 - Messwert (Schalterstellungen)
- b) Erweitern Sie den Ringspeicher der vorhergehenden Aufgabe zu einer doppelt verketteten Liste. Nach dem Ende der Messdatenerfassung sollen die Daten ausgegeben werden, der neueste Messwert zuerst.
- c) Das niedrigstwertige Bit hat nun eine spezielle Bedeutung. Wenn der Schalter einen High-Pegel liefert, soll der Ringspeicher halbiert werden (5 Elemente). Die neusten Messwerte sollen dabei erhalten bleiben. Bei Low-Pegel wird der Ringspeicher wieder auf 10 Elemente erweitert.

Aufgabe 7

Am Controller-Aufbau finden Sie ein analoges Potentiometer. Die abgegriffene Spannung können Sie mit dem Analog-Eingang `ADC1_IN5` (Pin 0 von `GPIO_B`) einlesen. Konfigurieren und programmieren Sie das Analog-Eingangsmodul so, dass Sie die Spannung messen können.

- a) Geben Sie die gelieferten Float-Wert auf dem Bildschirm aus.
- b) Stellen Sie die gelieferten Werte auf den Leuchtdioden dar. Bedenken Sie dabei, dass die rote LED-Reihe invertiert ist.

Aufgabe 8

Geräte mit Lautstärkeeinstellung (z.B. Autoradios) haben häufig ein digitales Potentiometer ohne Anschlag (Drehgeber). Am Praktikumsaufbau ist ein Drehgeber an GPIO_A, Pin 0 und 1 angeschlossen. Konfigurieren Sie den Drehgeber so, dass Sie auf die dadurch gelieferten Werte zugreifen können.

- a) Geben Sie die vom Drehgeber gelieferten Werte auf der Konsole aus.
- b) Stellen Sie die vom Drehgeber gelieferten Werte auf den Leuchtdioden dar.
- c) Bei einer Lautstärkeeinstellung ist der Zähler ein Maß für die Lautstärke. Geben Sie den Zähler als Balkenanzeige auf den LEDs aus.

Aufgabe 9

Ein Produktionsmodell bestehend aus drei Förderbändern mit unterschiedlichen Bearbeitungsstationen soll mit STM32 Nucleo-Systemen gesteuert werden. Jedem Förderband (Transferstrecke) ist ein Mikrocontroller zugeordnet. Die Tabelle der Sensoren und Aktoren, die Sie an den Bändern und Bearbeitungsstationen vorfinden, sehen sie in der Datei xy. Die genauen Unterlagen des Herstellers FESTO finden Sie ebenfalls im Handout.

Bestimmen Sie zunächst den aktiven Zustand der Sensoren. Programmieren Sie einen vereinfachten Ablauf:

- Werkstücke werden in regelmäßigem Abstand vom Magazin auf das Band geschoben (natürlich nur, wenn sich kein Werkstück vom Rücklauf vor dem Magazin befindet).
- Jedes Werkstück soll gedreht werden. Darf das Band dabei weiter laufen? Was ist mit den anderen Bändern? Mithilfe des optischen Sensors soll erkannt werden, ob ein RFID-Chip vorhanden ist (dunkle Oberfläche).
- Jedes zweite Werkstück wird auf die Rutsche ausgeschleust. Was passiert, wenn kein Ausschleusen mehr möglich ist, weil die Rutsche voll ist?
- Über den Rücklauf wird das Werkstück wieder auf das linke Band transportiert.
- Nutzen Sie Ampeln zur Anzeige: Es befindet sich ein Werkstück auf dem Band, das Magazin ist leer, die Rutsche ist voll, RFID-Chip erkannt usw.

Die Kommunikation der Controller untereinander und mit den RFID-Schreib-Lesestationen soll im Praktikum PE noch nicht betrachtet werden. Erweitern Sie Ihr Programm: Es soll die Höhe gemessen und als Balkenanzeige auf der LED-Reihe ausgegeben werden. Bearbeiten Sie zunächst Aufgabe 9, um den Analog-Input zu verstehen.